Weather Station System FreeRTOS (WIN32–MSVC)

Overview

This document details the implementation of a FreeRTOS-based Weather Station System developed for the WIN32–MSVC FreeRTOS simulator. The system consists of four concurrently running tasks responsible for simulated temperature monitoring, humidity monitoring, data logging, and real-time console display. The application demonstrates key principles of embedded systems including scheduling, multitasking, synchronisation, and deterministic timing.

1. System Requirements

• Temperature Monitoring Task — Reads simulated temperature every 5 seconds and stores it in a global variable.

• Humidity Monitoring Task — Reads simulated humidity every 7 seconds and stores it in a global variable.

• Data Logging Task — Logs timestamped readings every 10 seconds to weather_log.txt.

• Display Task — Outputs temperature and humidity to console every 2 seconds.

• Shared variables protected using a FreeRTOS mutex.

2. System Architecture

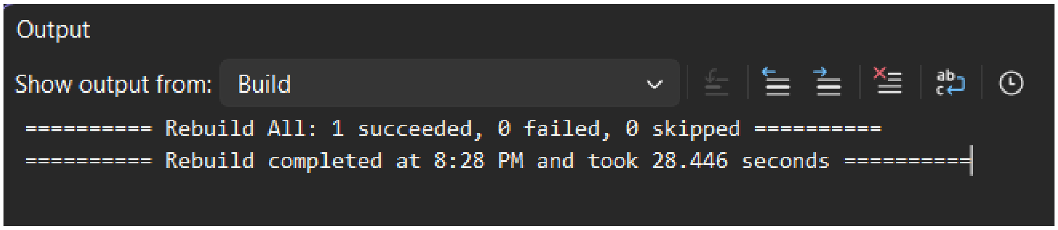
The system is structured around four periodic FreeRTOS tasks, each operating at a defined interval. A mutex ensures safe access to the shared temperature and humidity variables. The WIN32–MSVC FreeRTOS port runs tasks as Windows threads, enabling accurate timing simulation for development and debugging.

3. Print Preview — Screenshot Placeholders with Explanations

Below are placeholders for the screenshots required to demonstrate execution and validation of the system. Each placeholder is accompanied by an explanatory paragraph describing what should appear in the screenshot.

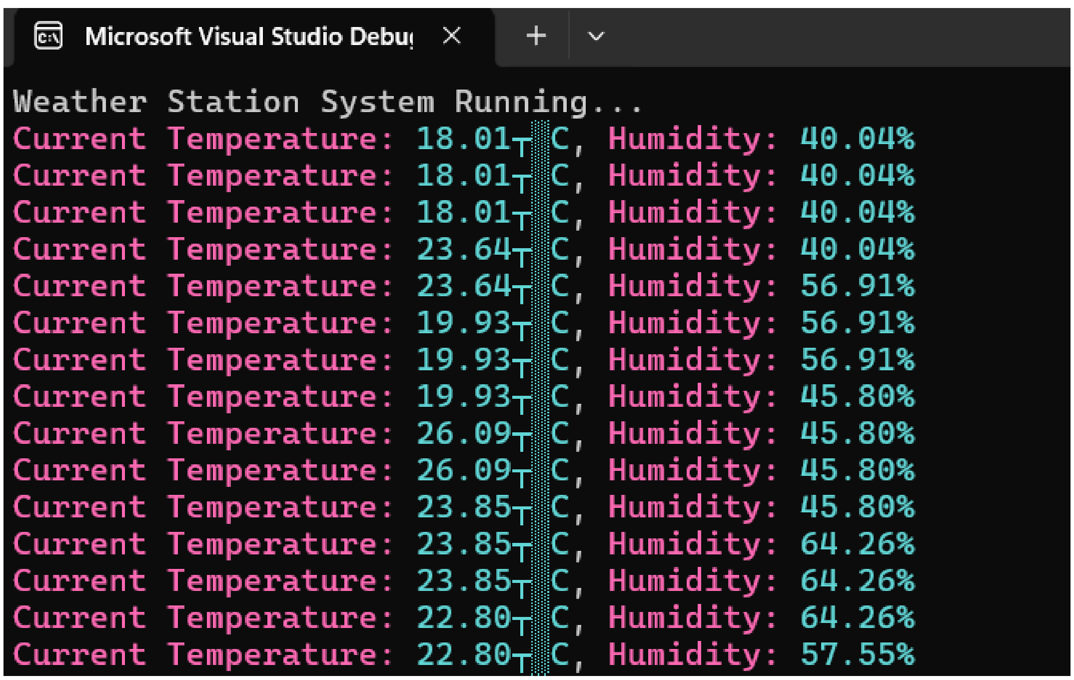
Visual Studio Build Window

This screenshot should display Visual Studio with a successful build of the WIN32–MSVC FreeRTOS Weather Station project. It verifies that the main.c file is correctly integrated, all required FreeRTOS kernel files are included, and the FreeRTOS scheduler is ready to run the system tasks.



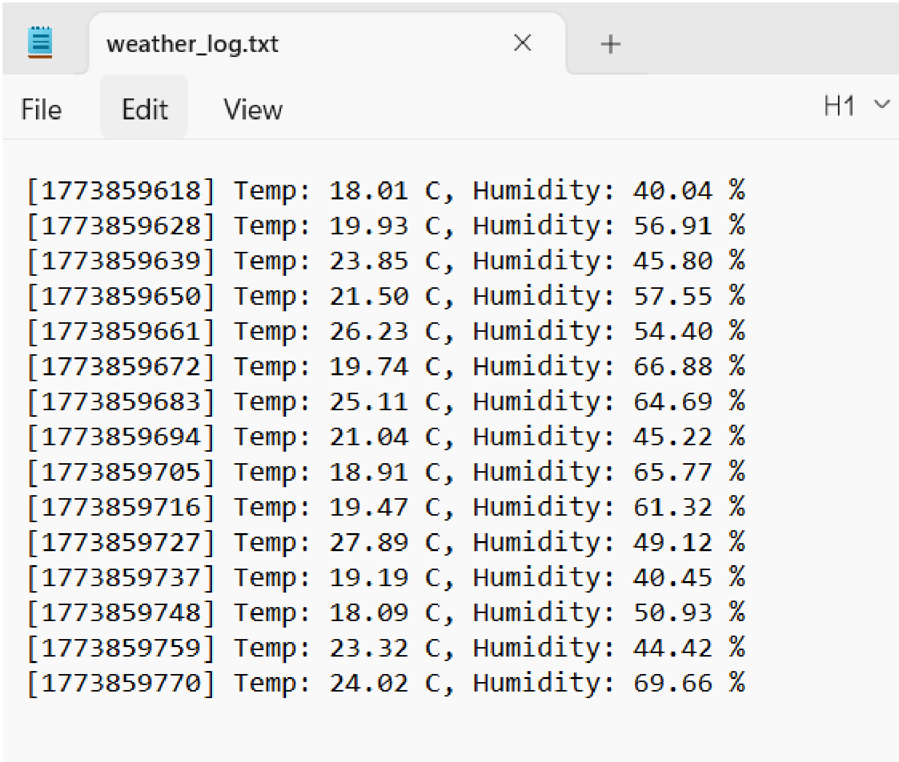
Console Output (Temperature & Humidity Updates)

This screenshot must show the running console output with continuously updating temperature and humidity readings. The values should refresh every 2 seconds, generated using pseudo-random simulation to represent sensor data. The presence of coloured output (pink for temperature and cyan for humidity) confirms successful operation of the Display Task and correct synchronisation.



weather_log.txt (Logged Timestamped Data)

This screenshot should show the contents of weather_log.txt. Each entry must include a UNIX-style timestamp, the most recent temperature reading, and the most recent humidity reading. This verifies that the Data Logging Task is running every 10 seconds and writing data safely using mutex-protected shared variables.




```
[1773859618] Temp: 18.01 C, Humidity: 40.04 %
[1773859628] Temp: 19.93 C, Humidity: 56.91 %
[1773859639] Temp: 23.85 C, Humidity: 45.80 %
[1773859650] Temp: 21.50 C, Humidity: 57.55 %
[1773859661] Temp: 26.23 C, Humidity: 54.40 %
[1773859672] Temp: 19.74 C, Humidity: 66.88 %
[1773859683] Temp: 25.11 C, Humidity: 64.69 %
[1773859694] Temp: 21.04 C, Humidity: 45.22 %
[1773859705] Temp: 18.91 C, Humidity: 65.77 %
[1773859716] Temp: 19.47 C, Humidity: 61.32 %
[1773859727] Temp: 27.89 C, Humidity: 49.12 %
[1773859737] Temp: 19.19 C, Humidity: 40.45 %
[1773859748] Temp: 18.09 C, Humidity: 50.93 %
[1773859759] Temp: 23.32 C, Humidity: 44.42 %
[1773859770] Temp: 24.02 C, Humidity: 69.66 %
```

4. Task Descriptions

• Temperature Monitoring Task — Simulates temperature readings every 5 seconds.

• Humidity Monitoring Task — Simulates humidity readings every 7 seconds.

• Data Logging Task — Appends timestamped sensor data to weather_log.txt every 10 seconds.

• Display Task — Outputs the latest readings in real-time every 2 seconds.

5. Data Logging

The Weather Station logs sensor readings using fprintf() into weather_log.txt. Each log line includes a timestamp, temperature in °C, and humidity in %. Logging every 10 seconds ensures accurate and periodic data recording.

6. Integration & System Flow

At startup, heap_5 regions are initialised to meet FreeRTOS memory requirements. The mutex is created, followed by initialisation of all four tasks. FreeRTOS then manages task execution based on defined intervals. Each task performs its function independently while sharing data safely through synchronisation mechanisms.

7. Alignment with Peckol (2019)

This implementation aligns with James K. Peckol's *Embedded Systems: A Contemporary Design Tool* (2019), including concepts such as multitasking, concurrency, real-time scheduling, shared-resource protection, and deterministic system behaviour in cyber-physical systems.

Appendix — Submission Checklist

1) main.c — FreeRTOS Weather Station implementation

2) FreeRTOSConfig.h — WIN32–MSVC configuration

3) weather_log.txt — Logged dataset

4) Screenshots for Print Preview

5) Word report (this document)